AI & ML Lab

Subject Code: 23A31403

Python Programs for All Experiments

Each experiment contains the aim, the complete program and the output obtained when the program was run (Python 3, pandas, NumPy, scikit-learn, matplotlib).

List of Experiments

Experiment 1(a) - Write a Python program to implement a Pandas Series with labels.
Experiment 1(b) - Create a Pandas Series from a dictionary.
Experiment 1(c) - Create a Pandas DataFrame.
Experiment 1(d) - Write a program that uses the Pandas methods describe(), head(), tail() and info().
Experiment 2 - Use Pandas inbuilt visualization to plot bar plots, histograms, line plots and scatter plots.
Experiment 3 - Write a program to implement Breadth First Search (BFS) using Python.
Experiment 4 - Write a program to implement the Best First Search algorithm.
Experiment 5 - Write a program to implement Depth First Search (DFS) using Python.
Experiment 6 - Write a program to implement Heuristic Search (8-puzzle with misplaced-tiles and Manhattan-distance heuristics).
Experiment 7 - Write a Python program to implement the A* and AO* algorithms (shortest path / AND-OR graph).
Experiment 8 - Apply pre-processing techniques to a dataset: attribute selection, handling missing values, discretization and elimination of outliers.
Experiment 9 - Apply the KNN algorithm for classification and regression.
Experiment 10 - Demonstrate the decision tree algorithm for a classification problem and perform parameter tuning for better results.
Experiment 11 - Apply the Random Forest algorithm for classification and regression.
Experiment 12 - Demonstrate the Naive Bayes classification algorithm.
Experiment 13 - Apply the Support Vector algorithm for classification.
Experiment 14 - Implement the K-means algorithm and apply it to a dataset. Evaluate performance by the sum of Euclidean distances of each example from its cluster centre, and test the performance as a function of K.

Libraries required: pip install pandas numpy matplotlib scikit-learn

Experiment 1(a)

Aim: Write a Python program to implement a Pandas Series with labels.

Program

```
# Experiment 1(a): Pandas Series with labels
import pandas as pd

marks = [85, 92, 78, 90, 66]
subjects = ['Maths', 'Physics', 'Chemistry', 'English', 'AI']

# Series with custom labels (index)
s = pd.Series(marks, index=subjects, name='Marks')
print("Series with labels:")
print(s)

print("\nMarks in Physics        :", s['Physics'])      # access by label
print("First element (by position):", s.iloc[0])       # access by position
print("\nSelected labels (Maths, AI):")
print(s[['Maths', 'AI']])

print("\nSubjects with marks > 80:")
print(s[s > 80])

print("\nMean marks:", s.mean())
print("Index     :", list(s.index))
print("Values    :", s.values)
```

Output

```
Series with labels:
Maths        85
Physics      92
Chemistry    78
English      90
AI           66
Name: Marks, dtype: int64

Marks in Physics        : 92
First element (by position): 85

Selected labels (Maths, AI):
Maths    85
AI       66
Name: Marks, dtype: int64

Subjects with marks > 80:
Maths      85
Physics    92
English    90
Name: Marks, dtype: int64

Mean marks: 82.2
```

```
Index     : ['Maths', 'Physics', 'Chemistry', 'English', 'AI']
Values    : [85 92 78 90 66]
```

Experiment 1(b)

Aim: Create a Pandas Series from a dictionary.

Program

```
# Experiment 1(b): Creating a Pandas Series from a dictionary
import pandas as pd

population = {'India': 1428, 'China': 1425, 'USA': 339, 'Indonesia': 277}

# Dictionary keys become the index, values become the data
s = pd.Series(population)
print("Series created from dictionary:")
print(s)

# Choose only some keys / change order using the index argument
s2 = pd.Series(population, index=['USA', 'India', 'Japan'])
print("\nSeries with selected keys (missing key gives NaN):")
print(s2)

print("\nPopulation of India:", s['India'])
print("Data type:", s.dtype)
print("Sorted by value:")
print(s.sort_values(ascending=False))
```

Output

```
Series created from dictionary:
India        1428
China        1425
USA           339
Indonesia     277
dtype: int64

Series with selected keys (missing key gives NaN):
USA       339.0
India    1428.0
Japan       NaN
dtype: float64

Population of India: 1428
Data type: int64
Sorted by value:
India        1428
China        1425
USA           339
Indonesia     277
dtype: int64
```

Experiment 1(c)

Aim: Create a Pandas DataFrame.

Program

```
# Experiment 1(c): Creating a Pandas DataFrame
import pandas as pd

# 1. From a dictionary of lists
data = {
    'Name': ['Asha', 'Ravi', 'Meena', 'Kiran'],
    'Age': [19, 20, 19, 21],
    'Dept': ['CSE', 'AI&ML', 'CSE', 'ECE'],
    'CGPA': [8.7, 9.1, 7.9, 8.2]
}
df = pd.DataFrame(data)
print("DataFrame from dictionary of lists:")
print(df)

# 2. From a list of lists with column names
rows = [['Asha', 19], ['Ravi', 20], ['Meena', 19]]
df2 = pd.DataFrame(rows, columns=['Name', 'Age'])
print("\nDataFrame from list of lists:")
print(df2)

# 3. From a list of dictionaries with custom row labels
records = [{'Name': 'Asha', 'Marks': 88}, {'Name': 'Ravi', 'Marks': 92}]
df3 = pd.DataFrame(records, index=['S1', 'S2'])
print("\nDataFrame from list of dictionaries:")
print(df3)

# Basic attributes
print("\nShape   :", df.shape)
print("Columns :", list(df.columns))
print("Index   :", list(df.index))
print("Dtypes  :\n", df.dtypes)
print("\nColumn 'CGPA':\n", df['CGPA'])
```

Output

```
DataFrame from dictionary of lists:
    Name  Age   Dept  CGPA
0   Asha   19    CSE   8.7
1   Ravi   20  AI&ML   9.1
2  Meena   19    CSE   7.9
3  Kiran   21    ECE   8.2

DataFrame from list of lists:
    Name  Age
0   Asha   19
1   Ravi   20
2  Meena   19
DataFrame from list of dictionaries:
```

```
    Name  Marks
S1  Asha     88
S2  Ravi     92

Shape   : (4, 4)
Columns : ['Name', 'Age', 'Dept', 'CGPA']
Index   : [0, 1, 2, 3]
Dtypes  :
 Name        str
Age       int64
Dept        str
CGPA    float64
dtype: object

Column 'CGPA':
 0    8.7
1    9.1
2    7.9
3    8.2
Name: CGPA, dtype: float64
```

Experiment 1(d)

Aim: Write a program that uses the Pandas methods describe(), head(), tail() and info().

Program

```
# Experiment 1(d): describe(), head(), tail(), info()
import pandas as pd

data = {
    'Name': ['Asha', 'Ravi', 'Meena', 'Kiran', 'Sita', 'Arjun',
             'Divya', 'Kumar', 'Latha', 'Mohan'],
    'Age': [19, 20, 19, 21, 20, 22, 19, 20, 21, 20],
    'Marks': [88, 92, 79, 65, 90, 73, 85, 95, 70, 81],
    'Dept': ['CSE', 'AI&ML', 'CSE', 'ECE', 'AI&ML', 'EEE',
             'CSE', 'AI&ML', 'ECE', 'CSE']
}
df = pd.DataFrame(data)

print("i) describe() - summary statistics of numeric columns")
print(df.describe())

print("\nii) head() - first 5 rows")
print(df.head())

print("\nhead(3) - first 3 rows")
print(df.head(3))

print("\niii) tail() - last 5 rows")
print(df.tail())

print("\ntail(2) - last 2 rows")
print(df.tail(2))

print("\niv) info() - structure of the DataFrame")
df.info()
```

Output

```
i) describe() - summary statistics of numeric columns
             Age      Marks
count  10.000000  10.000000
mean   20.100000  81.800000
std     0.994429  10.008885
min    19.000000  65.000000
25%    19.250000  74.500000
50%    20.000000  83.000000
75%    20.750000  89.500000
max    22.000000  95.000000

ii) head() - first 5 rows
    Name  Age  Marks   Dept
0   Asha   19     88    CSE
1   Ravi   20     92  AI&ML
```

```
2  Meena   19     79    CSE
3  Kiran   21     65    ECE
4   Sita   20     90  AI&ML

head(3) - first 3 rows
    Name  Age  Marks   Dept
0   Asha   19     88    CSE
1   Ravi   20     92  AI&ML
2  Meena   19     79    CSE

iii) tail() - last 5 rows
    Name  Age  Marks   Dept
5  Arjun   22     73    EEE
6  Divya   19     85    CSE
7  Kumar   20     95  AI&ML
8  Latha   21     70    ECE
9  Mohan   20     81    CSE

tail(2) - last 2 rows
    Name  Age  Marks Dept
8  Latha   21     70  ECE
9  Mohan   20     81  CSE

iv) info() - structure of the DataFrame
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    10 non-null     str
 1   Age     10 non-null     int64
 2   Marks   10 non-null     int64
 3   Dept    10 non-null     str
dtypes: int64(2), str(2)
memory usage: 452.0 bytes
```

Experiment 2

Aim: Use Pandas inbuilt visualization to plot bar plots, histograms, line plots and scatter plots.

Program

```
# Experiment 2: Pandas inbuilt visualization
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
              'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
    'Sales': [250, 270, 300, 280, 350, 400, 380, 420, 390, 450, 500, 530],
    'Profit': [40, 45, 55, 48, 70, 85, 78, 90, 80, 100, 115, 125]
}
df = pd.DataFrame(data)

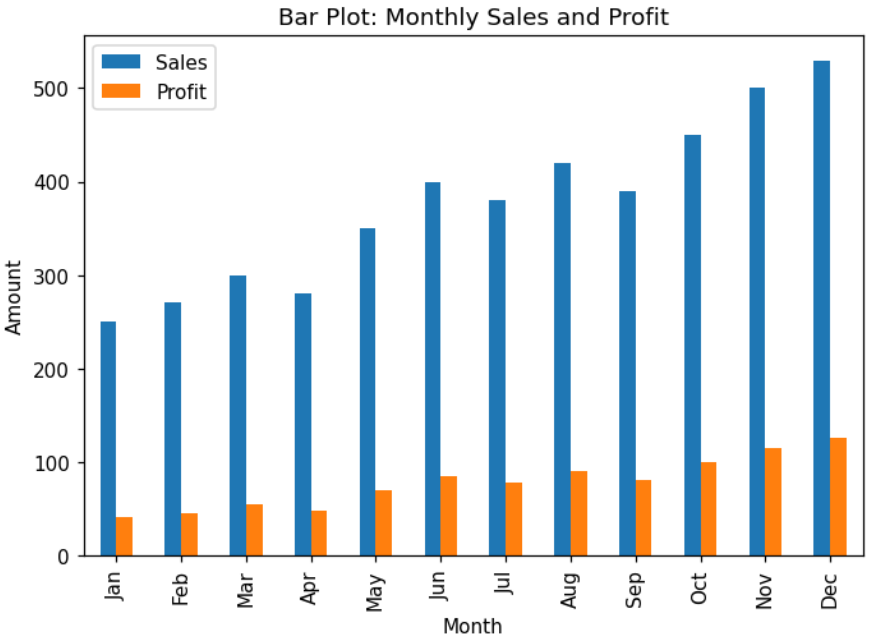
# i. Bar plot
df.plot.bar(x='Month', y=['Sales', 'Profit'],
            title='Bar Plot: Monthly Sales and Profit')
plt.ylabel('Amount')
plt.tight_layout()
plt.show()

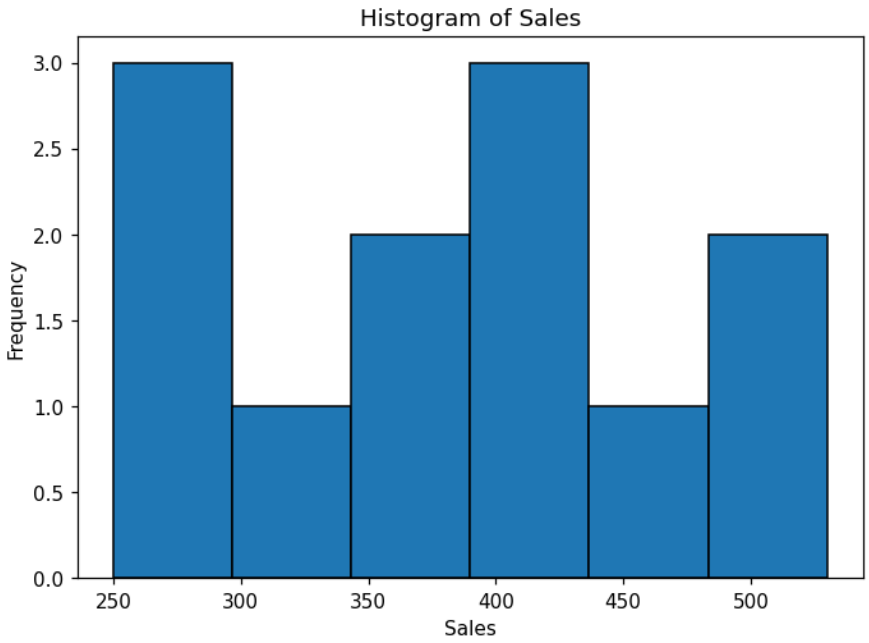
# ii. Histogram
df['Sales'].plot.hist(bins=6, edgecolor='black', title='Histogram of Sales')
plt.xlabel('Sales')
plt.tight_layout()
plt.show()

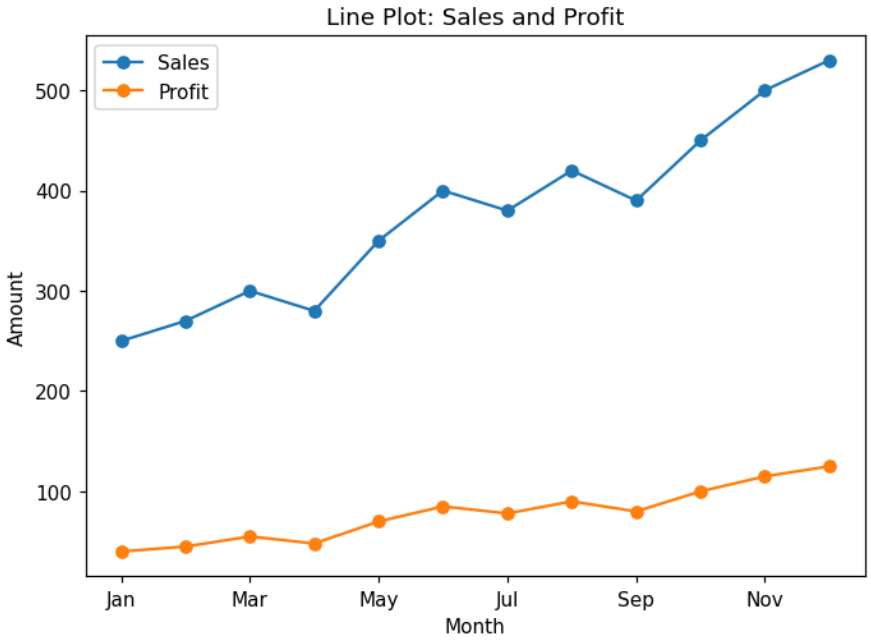
# iii. Line plot
df.plot.line(x='Month', y=['Sales', 'Profit'], marker='o',
             title='Line Plot: Sales and Profit')
plt.ylabel('Amount')
plt.tight_layout()
plt.show()

# iv. Scatter plot
df.plot.scatter(x='Sales', y='Profit', color='red',
                title='Scatter Plot: Sales vs Profit')
plt.tight_layout()
plt.show()
```

Output


Bar Plot: Monthly Sales and Profit
Sales
Profit
Amount
0
100
200
300
400
500
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Month


Histogram of Sales
Frequency
0.0
0.5
1.0
1.5
2.0
2.5
3.0
250
300
350
400
450
500
Sales


Line Plot: Sales and Profit
Sales
Profit
Amount
500
400
300
200
100
Jan
Mar
May
Jul
Sep
Nov
Month


Scatter Plot: Sales vs Profit
Profit
120
100
80
60
40
250
300
350
400
450
500
Sales

Experiment 3

Aim: Write a program to implement Breadth First Search (BFS) using Python.

Program

```
# Experiment 3: Breadth First Search (BFS)
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start):
    """Return the nodes in the order BFS visits them."""
    visited = [start]
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()          # FIFO queue
        order.append(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)
    return order

def bfs_shortest_path(graph, start, goal):
    """Return the path with the fewest edges from start to goal."""
    queue = deque([[start]])
    visited = {start}
    while queue:
        path = queue.popleft()
        node = path[-1]
        if node == goal:
            return path
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(path + [neighbour])
    return None

print("BFS traversal from A :", bfs(graph, 'A'))
print("Shortest path A -> F :", bfs_shortest_path(graph, 'A', 'F'))
```

Output

```
BFS traversal from A : ['A', 'B', 'C', 'D', 'E', 'F']
Shortest path A -> F : ['A', 'C', 'F']
```

Experiment 4

Aim: Write a program to implement the Best First Search algorithm.

Program

```
# Experiment 4: Best First Search (greedy)
# Uses a priority queue ordered by the heuristic value h(n)
import heapq

graph = {
    'S': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['E', 'F'],
    'C': [], 'D': [],
    'E': ['H'],
    'F': ['I', 'G'],
    'H': [], 'I': [], 'G': []
}
# Heuristic value h(n): estimated cost from node n to the goal
h = {'S': 13, 'A': 12, 'B': 4, 'C': 7, 'D': 3,
     'E': 8, 'F': 2, 'H': 4, 'I': 9, 'G': 0}

def best_first_search(start, goal):
    open_list = [(h[start], start, [start])]    # (h value, node, path)
    visited = set()
    while open_list:
        hv, node, path = heapq.heappop(open_list)   # node with smallest h
        if node in visited:
            continue
        visited.add(node)
        print(f"Expanding {node}  (h = {hv})")
        if node == goal:
            return path
        for neighbour in graph[node]:
            if neighbour not in visited:
                heapq.heappush(open_list,
                               (h[neighbour], neighbour, path + [neighbour]))
    return None

path = best_first_search('S', 'G')
print("Path found:", ' -> '.join(path))
```

Output

```
Expanding S  (h = 13)
Expanding B  (h = 4)
Expanding F  (h = 2)
Expanding G  (h = 0)
Path found: S -> B -> F -> G
```

Experiment 5

Aim: Write a program to implement Depth First Search (DFS) using Python.

Program

```
# Experiment 5: Depth First Search (DFS)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Recursive DFS
def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = []
    visited.append(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs_recursive(graph, neighbour, visited)
    return visited

# Iterative DFS using an explicit stack (LIFO)
def dfs_iterative(graph, start):
    visited = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.append(node)
            # push neighbours in reverse so the left-most is explored first
            for neighbour in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append(neighbour)
    return visited

# DFS that returns a path to a goal
def dfs_path(graph, start, goal, path=None):
    path = (path or []) + [start]
    if start == goal:
        return path
    for neighbour in graph[start]:
        if neighbour not in path:
            result = dfs_path(graph, neighbour, goal, path)
            if result:
                return result
    return None

print("DFS (recursive) from A :", dfs_recursive(graph, 'A'))
```

```
print("DFS (iterative) from A :", dfs_iterative(graph, 'A'))
print("A path from A to F     :", dfs_path(graph, 'A', 'F'))
```

Output

```
DFS (recursive) from A : ['A', 'B', 'D', 'E', 'F', 'C']
DFS (iterative) from A : ['A', 'B', 'D', 'E', 'F', 'C']
A path from A to F     : ['A', 'B', 'E', 'F']
```

Experiment 6

Aim: Write a program to implement Heuristic Search (8-puzzle with misplaced-tiles and Manhattan-distance heuristics).

Program

```
# Experiment 6: Heuristic search - solving the 8-puzzle with heuristic functions
import heapq

GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)          # 0 represents the blank tile

def misplaced_tiles(state):
    """h1(n): number of tiles that are not in their goal position."""
    return sum(1 for i, t in enumerate(state) if t != 0 and t != GOAL[i])

def manhattan(state):
    """h2(n): sum of Manhattan distances of tiles from their goal positions."""
    total = 0
    for i, t in enumerate(state):
        if t != 0:
            g = GOAL.index(t)
            total += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return total

def neighbours(state):
    i = state.index(0)
    r, c = divmod(i, 3)
    result = []
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            s = list(state)
            s[i], s[j] = s[j], s[i]
            result.append(tuple(s))
    return result

def heuristic_search(start, h):
    """Always expand the state with the lowest heuristic value."""
    open_list = [(h(start), start, [start])]
    visited = {start}
    expanded = 0
    while open_list:
        _, state, path = heapq.heappop(open_list)
        expanded += 1
        if state == GOAL:
            return path, expanded
        for nb in neighbours(state):
            if nb not in visited:
                visited.add(nb)
```

```
                heapq.heappush(open_list, (h(nb), nb, path + [nb]))
    return None, expanded

def show(state):
    for r in range(3):
        print(' '.join(str(x) if x else '_' for x in state[r * 3:r * 3 + 3]))

start = (4, 1, 3,
         7, 2, 6,
         0, 5, 8)

heuristics = [("Misplaced tiles", misplaced_tiles),
              ("Manhattan distance", manhattan)]
for name, h in heuristics:
    path, expanded = heuristic_search(start, h)
    print(f"Heuristic: {name}")
    print(f"  Moves in solution : {len(path) - 1}")
    print(f"  States expanded   : {expanded}")

print("\nSolution path (Manhattan distance):")
path, _ = heuristic_search(start, manhattan)
for step, state in enumerate(path):
    print(f"Step {step}:")
    show(state)
```

Output

```
Heuristic: Misplaced tiles
  Moves in solution : 6
  States expanded   : 7
Heuristic: Manhattan distance
  Moves in solution : 6
  States expanded   : 7

Solution path (Manhattan distance):
Step 0:
4 1 3
7 2 6
_ 5 8
Step 1:
4 1 3
_ 2 6
7 5 8
Step 2:
_ 1 3
4 2 6
7 5 8
Step 3:
1 _ 3
4 2 6
7 5 8
Step 4:
1 2 3
```

```
4 _ 6
7 5 8
Step 5:
1 2 3
4 5 6
7 _ 8
Step 6:
1 2 3
4 5 6
7 8 _
```

Experiment 7

Aim: Write a Python program to implement the A* and AO* algorithms (shortest path / AND-OR graph).

Program

```
# Experiment 7: A* and AO* algorithms
import heapq

# ---------------------------------------------------------------
# Part 1: A* algorithm - shortest path in a weighted graph
# f(n) = g(n) + h(n)
# ---------------------------------------------------------------
graph = {
    'S': {'A': 1, 'B': 4},
    'A': {'B': 2, 'C': 5, 'D': 12},
    'B': {'C': 2},
    'C': {'D': 3, 'G': 7},
    'D': {'G': 2},
    'G': {}
}
h = {'S': 7, 'A': 6, 'B': 4, 'C': 4, 'D': 2, 'G': 0}   # admissible heuristic

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]     # (f, g, node, path)
    best_g = {start: 0}
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if node == goal:
            return path, g
        if g > best_g.get(node, float('inf')):
            continue
        for neighbour, cost in graph[node].items():
            new_g = g + cost
            if new_g < best_g.get(neighbour, float('inf')):
                best_g[neighbour] = new_g
                heapq.heappush(open_list, (new_g + h[neighbour], new_g,
                                           neighbour, path + [neighbour]))
    return None, float('inf')

path, cost = a_star('S', 'G')
print("A* shortest path :", ' -> '.join(path))
print("A* path cost     :", cost)

# ---------------------------------------------------------------
# Part 2: AO* algorithm - AND-OR graph
# Each node has a list of alternatives (OR); every alternative is a
# list of (child, edge_cost) pairs that must ALL be solved (AND).
# Nodes that do not appear in `ao_graph` are terminal; their cost is H[node].
# ---------------------------------------------------------------
H = {'A': 1, 'B': 6, 'C': 2, 'D': 12, 'E': 2, 'F': 1,
     'G': 5, 'H': 7, 'I': 7, 'J': 1}
```

```
ao_graph = {
    'A': [[('B', 1), ('C', 1)], [('D', 1)]],    # (B AND C) OR D
    'B': [[('G', 1)], [('H', 1)]],              # G OR H
    'C': [[('J', 1)]],
    'D': [[('E', 1), ('F', 1)]],                # E AND F
    'G': [[('I', 1)]]
}

cost = {}          # revised (minimum) cost of every node
best_choice = {}   # best alternative selected for every non-terminal node

def ao_star(node):
    if node in cost:
        return cost[node]
    if node not in ao_graph:                    # terminal node
        cost[node] = H[node]
        return cost[node]
    best = float('inf')
    for option in ao_graph[node]:
        total = sum(w + ao_star(child) for child, w in option)
        if total < best:
            best, best_choice[node] = total, option
    cost[node] = best                           # revise the cost of the node
    return best

def print_solution(node):
    if node in best_choice:
        children = best_choice[node]
        print(f"{node} -> " + ' AND '.join(c for c, _ in children))
        for child, _ in children:
            print_solution(child)

print("\nAO* minimum cost of solving A:", ao_star('A'))
print("AO* solution graph:")
print_solution('A')
```

Output

```
A* shortest path : S -> A -> B -> C -> D -> G
A* path cost     : 10

AO* minimum cost of solving A: 6
AO* solution graph:
A -> D
D -> E AND F
```

Experiment 8

Aim: Apply pre-processing techniques to a dataset: attribute selection, handling missing values, discretization and elimination of outliers.

Program

```
# Experiment 8: Data pre-processing techniques
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, f_classif, VarianceThreshold
from sklearn.preprocessing import KBinsDiscretizer

iris = load_iris(as_frame=True)
clean = iris.frame.copy()                        # original clean data (150 rows)
features = list(iris.feature_names)

# ---------------- a. Attribute (feature) selection ----------------
print("a. ATTRIBUTE SELECTION")
X, y = clean[features], clean['target']
selector = SelectKBest(score_func=f_classif, k=2)   # keep 2 best attributes
selector.fit(X, y)
scores = pd.Series(selector.scores_, index=features)
scores = scores.sort_values(ascending=False)
print("ANOVA F-scores:\n", scores.round(2))
selected = list(X.columns[selector.get_support()])
print("Selected attributes:", selected)

vt = VarianceThreshold(threshold=0.2)      # drop low-variance attributes
vt.fit(X)
print("Kept by variance threshold:", list(X.columns[vt.get_support()]))

# Create a "dirty" copy with missing values and outliers for the next steps
rng = np.random.default_rng(42)
dirty = clean.copy()
for col in features:
    idx = rng.choice(dirty.index, 8, replace=False)
    dirty.loc[idx, col] = np.nan
dirty.loc[10, 'sepal width (cm)'] = 15.0               # outliers
dirty.loc[20, 'petal length (cm)'] = 40.0
dirty.loc[30, 'sepal length (cm)'] = 60.0

# ---------------- b. Handling missing values ----------------
print("\nb. HANDLING MISSING VALUES")
print("Missing values per column:\n", dirty.isnull().sum())

filled = dirty.copy()
# median imputation
filled[features] = filled[features].fillna(filled[features].median())
print("Missing after imputation:", int(filled.isnull().sum().sum()))
dropped = dirty.dropna()                           # alternative: drop rows
print("Rows before dropping:", len(dirty), " after dropna():", len(dropped))
```

```
# ---------------- c. Discretization ----------------
print("\nc. DISCRETIZATION")
filled['petal_length_bin'] = pd.cut(filled['petal length (cm)'],
                                    bins=[0, 2, 5, 100],
                                    labels=['short', 'medium', 'long'])
print("Equal-interval/custom bins (pd.cut):")
print(filled['petal_length_bin'].value_counts())

kbd = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='quantile')
clean_cols = ['sepal width (cm)']
filled['sepal_width_bin'] = kbd.fit_transform(filled[clean_cols]).astype(int)
print("\nQuantile bins (KBinsDiscretizer):")
print(filled['sepal_width_bin'].value_counts().sort_index())

# ---------------- d. Elimination of outliers ----------------
print("\nd. ELIMINATION OF OUTLIERS (IQR method)")
data = filled[features]
Q1, Q3 = data.quantile(0.25), data.quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
is_outlier = ((data < lower) | (data > upper)).any(axis=1)
print("Number of outlier rows detected:", int(is_outlier.sum()))
no_outliers = filled[~is_outlier]
print("Rows before:", len(filled), " after:", len(no_outliers))

z = (data - data.mean()) / data.std()                  # Z-score method
print("Outlier rows by Z-score:", int((z.abs() > 3).any(axis=1).sum()))
```

Output

```
a. ATTRIBUTE SELECTION
ANOVA F-scores:
 petal length (cm)    1180.16
petal width (cm)      960.01
sepal length (cm)     119.26
sepal width (cm)       49.16
dtype: float64
Selected attributes: ['petal length (cm)', 'petal width (cm)']
Kept by variance threshold: ['sepal length (cm)', 'petal length (cm)', 'petal
    width (cm)']

b. HANDLING MISSING VALUES
Missing values per column:
 sepal length (cm)    8
sepal width (cm)     8
petal length (cm)    8
petal width (cm)     8
target               0
dtype: int64
Missing after imputation: 0
Rows before dropping: 150  after dropna(): 122
c. DISCRETIZATION
```

```
Equal-interval/custom bins (pd.cut):
petal_length_bin
medium    63
short     46
long      41
Name: count, dtype: int64

Quantile bins (KBinsDiscretizer):
sepal_width_bin
0    44
1    52
2    54
Name: count, dtype: int64

d. ELIMINATION OF OUTLIERS (IQR method)
Number of outlier rows detected: 7
Rows before: 150  after: 143
Outlier rows by Z-score: 3
```

Experiment 9

Aim: Apply the KNN algorithm for classification and regression.

Program

```
# Experiment 9: KNN for classification and regression
import numpy as np
from sklearn.datasets import load_iris, load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, mean_squared_error,
                             r2_score)

# ---------------- KNN Classification (Iris) ----------------
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42, stratify=y)
scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

print("KNN CLASSIFICATION")
for k in [1, 3, 5, 7, 9, 11]:
    knn = KNeighborsClassifier(n_neighbors=k).fit(X_train_s, y_train)
    acc = accuracy_score(y_test, knn.predict(X_test_s))
    print(f"  k = {k:2d}  accuracy = {acc:.4f}")

knn = KNeighborsClassifier(n_neighbors=5).fit(X_train_s, y_train)
pred = knn.predict(X_test_s)
print("\nResults for k = 5")
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
names = load_iris().target_names
print(classification_report(y_test, pred, target_names=names))

# ---------------- KNN Regression (Diabetes) ----------------
X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)
scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

print("KNN REGRESSION")
for k in [1, 3, 5, 10, 15, 20]:
    reg = KNeighborsRegressor(n_neighbors=k).fit(X_train_s, y_train)
    p = reg.predict(X_test_s)
    rmse = np.sqrt(mean_squared_error(y_test, p))
    print(f"  k = {k:2d}  RMSE = {rmse:7.3f}  R2 = {r2_score(y_test, p):.4f}")
```

Output

```
KNN CLASSIFICATION
```

```
  k =  1  accuracy = 0.9333
  k =  3  accuracy = 0.9111
  k =  5  accuracy = 0.9111
  k =  7  accuracy = 0.9333
  k =  9  accuracy = 0.9556
  k = 11  accuracy = 0.9556

Results for k = 5
Accuracy: 0.9111
Confusion matrix:
 [[15  0  0]
 [ 0 15  0]
 [ 0  4 11]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.79      1.00      0.88        15
   virginica       1.00      0.73      0.85        15

    accuracy                           0.91        45
   macro avg       0.93      0.91      0.91        45
weighted avg       0.93      0.91      0.91        45

KNN REGRESSION
  k =  1  RMSE =  77.244  R2 = -0.1053
  k =  3  RMSE =  60.253  R2 = 0.3275
  k =  5  RMSE =  57.252  R2 = 0.3928
  k = 10  RMSE =  56.287  R2 = 0.4131
  k = 15  RMSE =  54.903  R2 = 0.4416
  k = 20  RMSE =  53.978  R2 = 0.4603
```

Experiment 10

Aim: Demonstrate the decision tree algorithm for a classification problem and perform parameter tuning for better results.

Program

```
# Experiment 10: Decision tree classification with parameter tuning
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report)

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42,
    stratify=data.target)

# Baseline tree with default parameters
base = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
print("Baseline tree  - train accuracy:",
      round(base.score(X_train, y_train), 4))
print("Baseline tree  - test accuracy :",
      round(accuracy_score(y_test, base.predict(X_test)), 4))

# Parameter tuning using GridSearchCV (5-fold cross validation)
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [2, 3, 4, 5, 6, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
grid = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid,
                    cv=5, scoring='accuracy')
grid.fit(X_train, y_train)

print("\nBest parameters   :", grid.best_params_)
print("Best CV accuracy  :", round(grid.best_score_, 4))

best = grid.best_estimator_
pred = best.predict(X_test)
print("\nTuned tree     - test accuracy :",
      round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=data.target_names))

# Visualize the (top of the) tuned tree
plt.figure(figsize=(14, 7))
plot_tree(best, max_depth=3, feature_names=data.feature_names,
          class_names=data.target_names, filled=True, fontsize=7)
plt.title("Tuned Decision Tree (top 3 levels)")
```

```
plt.show()
```

Output

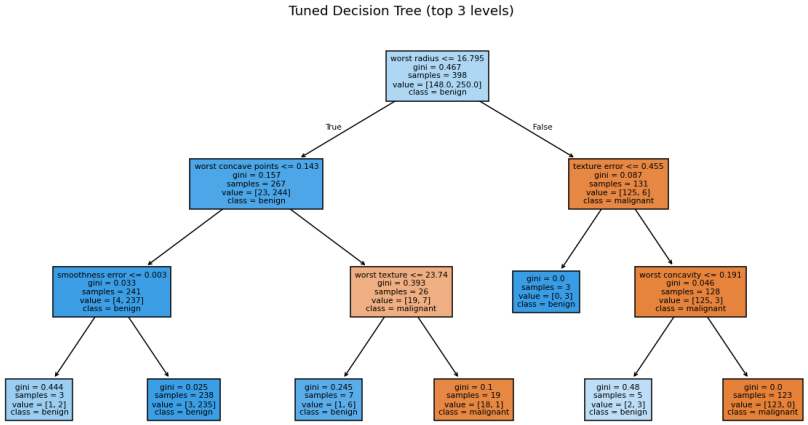
```
Baseline tree  - train accuracy: 1.0
Baseline tree  - test accuracy : 0.9181

Best parameters   : {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf':
    2, 'min_samples_split': 2}
Best CV accuracy  : 0.9449

Tuned tree     - test accuracy : 0.924
Confusion matrix:
 [[ 56   8]
 [  5 102]]
              precision    recall  f1-score   support

   malignant       0.92      0.88      0.90        64
      benign       0.93      0.95      0.94       107

    accuracy                           0.92       171
   macro avg       0.92      0.91      0.92       171
weighted avg       0.92      0.92      0.92       171
```



Experiment 11

Aim: Apply the Random Forest algorithm for classification and regression.

Program

```
# Experiment 11: Random Forest for classification and regression
import numpy as np
from sklearn.datasets import load_wine, load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             mean_squared_error, r2_score)

# ---------------- Random Forest Classification (Wine) ----------------
wine = load_wine()
X_train, X_test, y_train, y_test = train_test_split(
    wine.data, wine.target, test_size=0.3, random_state=42,
    stratify=wine.target)
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)
print("RANDOM FOREST CLASSIFICATION")
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))

imp = sorted(zip(wine.feature_names, clf.feature_importances_),
             key=lambda t: -t[1])
print("Top 5 important features:")
for name, score in imp[:5]:
    print(f"  {name:25s} {score:.4f}")

for n in [1, 10, 50, 100, 200]:
    m = RandomForestClassifier(n_estimators=n, random_state=42)
    m.fit(X_train, y_train)
    print(f"  n_estimators = {n:3d}  accuracy = {m.score(X_test, y_test):.4f}")

# ---------------- Random Forest Regression (Diabetes) ----------------
diab = load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(
    diab.data, diab.target, test_size=0.3, random_state=42)
reg = RandomForestRegressor(n_estimators=200, random_state=42)
reg.fit(X_train, y_train)
p = reg.predict(X_test)
print("\nRANDOM FOREST REGRESSION")
print("RMSE:", round(float(np.sqrt(mean_squared_error(y_test, p))), 3))
print("R2  :", round(r2_score(y_test, p), 4))
```

Output

```
RANDOM FOREST CLASSIFICATION
Accuracy: 1.0
Confusion matrix:
 [[18  0  0]
```

```
 [ 0 21  0]
 [ 0  0 15]]
Top 5 important features:
  alcohol                   0.1627
  color_intensity           0.1597
  flavanoids                0.1563
  proline                   0.1241
  hue                       0.1110
  n_estimators =   1  accuracy = 0.9259
  n_estimators =  10  accuracy = 0.9630
  n_estimators =  50  accuracy = 1.0000
  n_estimators = 100  accuracy = 1.0000
  n_estimators = 200  accuracy = 1.0000

RANDOM FOREST REGRESSION
RMSE: 52.964
R2  : 0.4803
```

Experiment 12

Aim: Demonstrate the Naive Bayes classification algorithm.

Program

```
# Experiment 12: Naive Bayes classification
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report)

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42,
    stratify=iris.target)

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Class prior probabilities:", np.round(model.class_prior_, 3))
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=iris.target_names))

# Probability of each class for the first three test samples
proba = model.predict_proba(X_test[:3])
for i, p in enumerate(proba):
    print(f"Sample {i + 1}: {dict(zip(iris.target_names, np.round(p, 4)))}"
          f"  -> predicted: {iris.target_names[pred[i]]}")

# Predict a new, unseen flower: [sepal len, sepal wid, petal len, petal wid]
new = [[5.9, 3.0, 5.1, 1.8]]
label = iris.target_names[model.predict(new)[0]]
print("\nPrediction for", new[0], "->", label)

scores = cross_val_score(GaussianNB(), iris.data, iris.target, cv=10)
print("10-fold CV accuracy: mean =", round(scores.mean(), 4),
      " std =", round(scores.std(), 4))
```

Output

```
Class prior probabilities: [0.333 0.333 0.333]
Accuracy: 0.9111
Confusion matrix:
 [[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
```

```
  versicolor       0.82      0.93      0.88        15
   virginica       0.92      0.80      0.86        15

    accuracy                           0.91        45
   macro avg       0.92      0.91      0.91        45
weighted avg       0.92      0.91      0.91        45

Sample 1: {np.str_('setosa'): np.float64(0.0), np.str_('versicolor'):
    np.float64(0.0), np.str_('virginica'): np.float64(1.0)}  -> predicted:
    virginica
Sample 2: {np.str_('setosa'): np.float64(0.0), np.str_('versicolor'):
    np.float64(0.9955), np.str_('virginica'): np.float64(0.0045)}  ->
    predicted: versicolor
Sample 3: {np.str_('setosa'): np.float64(0.0), np.str_('versicolor'):
    np.float64(0.8472), np.str_('virginica'): np.float64(0.1528)}  ->
    predicted: versicolor

Prediction for [5.9, 3.0, 5.1, 1.8] -> virginica
10-fold CV accuracy: mean = 0.9533  std = 0.0427
```

Experiment 13

Aim: Apply the Support Vector algorithm for classification.

Program

```
# Experiment 13: Support Vector Machine for classification
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report)

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42,
    stratify=data.target)

# Compare different kernels (feature scaling is important for SVM)
print("Kernel comparison:")
for kernel in ['linear', 'poly', 'rbf', 'sigmoid']:
    model = make_pipeline(StandardScaler(), SVC(kernel=kernel))
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    print(f"  kernel = {kernel:8s} test accuracy = {acc:.4f}")

# Tune C and gamma for the RBF kernel
pipe = make_pipeline(StandardScaler(), SVC(kernel='rbf'))
param_grid = {'svc__C': [0.1, 1, 10, 100],
              'svc__gamma': [0.001, 0.01, 0.1, 1]}
grid = GridSearchCV(pipe, param_grid, cv=5).fit(X_train, y_train)
print("\nBest parameters  :", grid.best_params_)
print("Best CV accuracy :", round(grid.best_score_, 4))

pred = grid.predict(X_test)
print("Test accuracy    :", round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=data.target_names))
```

Output

```
Kernel comparison:
  kernel = linear   test accuracy = 0.9825
  kernel = poly     test accuracy = 0.8947
  kernel = rbf      test accuracy = 0.9766
  kernel = sigmoid  test accuracy = 0.9357

Best parameters  : {'svc__C': 10, 'svc__gamma': 0.001}
Best CV accuracy : 0.9799
Test accuracy    : 0.9708
Confusion matrix:
 [[ 60   4]
```

```
[  1 106]]
              precision    recall  f1-score   support

   malignant       0.98      0.94      0.96        64
      benign       0.96      0.99      0.98       107

    accuracy                           0.97       171
   macro avg       0.97      0.96      0.97       171
weighted avg       0.97      0.97      0.97       171
```

Experiment 14

Aim: Implement the K-means algorithm and apply it to a dataset. Evaluate performance by the sum of Euclidean distances of each example from its cluster centre, and test the performance as a function of K.

Program

```
# Experiment 14: K-means clustering - performance as a function of K
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# data selected: Iris (150 examples, 4 attributes)
X = load_iris().data

def kmeans(X, k, max_iter=100, seed=0):
    rng = np.random.default_rng(seed)
    # random initial centres
    centers = X[rng.choice(len(X), k, replace=False)]
    for _ in range(max_iter):
        # assign every example to the nearest centre (Euclidean distance)
        dist = np.linalg.norm(X[:, None, :] - centers[None, :, :], axis=2)
        labels = dist.argmin(axis=1)
        # recompute each centre as the mean of its cluster
        new_centers = np.array([
            X[labels == j].mean(axis=0) if np.any(labels == j) else centers[j]
            for j in range(k)])
        if np.allclose(new_centers, centers):
            break
        centers = new_centers
    # performance: sum of Euclidean distances of each example
    # from its cluster centre
    total_distance = np.linalg.norm(X - centers[labels], axis=1).sum()
    return labels, centers, total_distance

def best_of_runs(X, k, runs=10):
    return min((kmeans(X, k, seed=s) for s in range(runs)), key=lambda r: r[2])

ks = range(1, 11)
results = {}
print(" K   Sum of Euclidean distances")
for k in ks:
    labels, centers, total = best_of_runs(X, k)
    results[k] = total
    print(f"{k:2d}   {total:10.3f}")

# Details for K = 3
labels, centers, total = best_of_runs(X, 3)
print("\nFor K = 3")
print("Cluster sizes:", np.bincount(labels))
print("Cluster centres:\n", np.round(centers, 3))
```

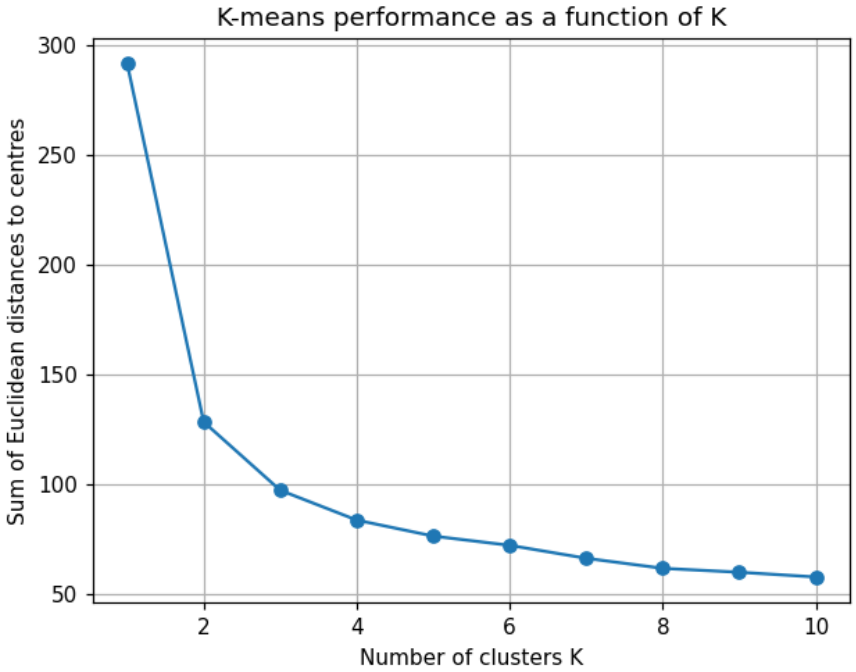
```
# Plot performance against K (elbow curve)
plt.plot(list(ks), list(results.values()), marker='o')
plt.xlabel('Number of clusters K')
plt.ylabel('Sum of Euclidean distances to centres')
plt.title('K-means performance as a function of K')
plt.grid(True)
plt.show()

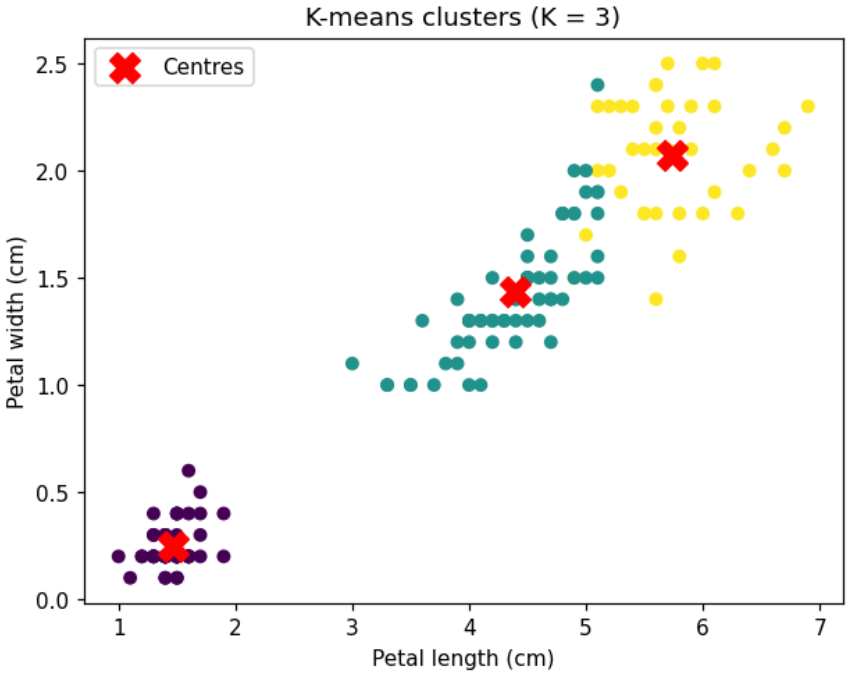
# Plot the clusters for K = 3 (using two attributes)
plt.scatter(X[:, 2], X[:, 3], c=labels, cmap='viridis', s=30)
plt.scatter(centers[:, 2], centers[:, 3], c='red', marker='X', s=200,
            label='Centres')
plt.xlabel('Petal length (cm)')
plt.ylabel('Petal width (cm)')
plt.title('K-means clusters (K = 3)')
plt.legend()
plt.show()
```

Output

```
 K   Sum of Euclidean distances
 1      291.610
 2      128.337
 3       97.205
 4       83.608
 5       76.316
 6       72.104
 7       66.205
 8       61.630
 9       59.856
10       57.686

For K = 3
Cluster sizes: [50 62 38]
Cluster centres:
 [[5.006 3.428 1.462 0.246]
 [5.902 2.748 4.394 1.434]
 [6.85  3.074 5.742 2.071]]
```


K-means performance as a function of K
Sum of Euclidean distances to centres
Number of clusters K
300
250
200
150
100
50
2
4
6
8
10


K-means clusters (K = 3)
Centres
Petal width (cm)
Petal length (cm)
2.5
2.0
1.5
1.0
0.5
0.0
1
2
3
4
5
6
7